

PSTAT100 Final Project Report

Group: Ethan Chuang, Matthew Kim, Edward Lee, Angela Pei, Matthew Tran

Date: 2026-05-06

Topic Outline:

- Final project report (Interactive Jupyter notebook):
 - Abstract, introduction, methodology, data preparation & EDA, results, conclusion, references.
- **Data:** UCI Adult / Census Income (`database/adult.data` , `adult.test`).
- **Outcome:** predict income **> \$50K** vs **<= \$50K** (binary classification).
- **Models:** logistic regression (`sklearn`) vs random forest comparison on the official train/test split.

1. Abstract

We study the **UCI Adult** extract (Becker and Kohavi, UCI repository), a widely used benchmark built from 1994 Census-related records, where each row describes an adult living in the United States together with a binary income label: **over \$50K per year** versus **at most \$50K**. The practical question is how well routine demographic and job-related fields predict that label in this fixed train and test split, not whether any single feature causes higher income.

We treat question marks in the raw files as missing values, summarize missingness and duplicates, and explore how quantitative predictors co-move with the outcome using plots and correlations in the spirit of the course exploratory data analysis labs. We then fit **logistic regression** inside a `sklearn` pipeline with median imputation for numeric fields, mode imputation for categoricals, scaling of numeric inputs, and one-hot encoding of categories. For comparison we fit a **random forest** on the same processed features.

On the official test partition, a naive baseline that always predicts the majority class **<=50K** reaches about **76.4%** accuracy. The logistic model reaches about **85.1%** accuracy and **0.904** ROC-AUC; the random forest is similar in accuracy on this split with slightly lower ROC-AUC in our run. Confusion matrices show where each model

confuses the minority **>50K** class with **<=50K**. Overall, the notebook documents an end-to-end data science workflow from messy survey-like inputs to evaluated classifiers, with limitations noted on causality, historical snapshot bias, and duplicate information across education fields.

2. Introduction

This report follows the course project scaffold: abstract (**Section 1**), roadmap here (**Section 2**), mathematical setup (**Section 3**), data sourcing and preparation (**Section 4**), estimation results (**Section 5**), synthesis (**Section 6**), and citations (**Section 7**).

The Adult prediction task is a standard illustration of **supervised classification**. Findings describe associations in a historical extract and should not be read as causal effects of any single variable.

3. Methodology

3.1 Problem formulation

Let $(y_i \in \{0,1\})$ encode whether record (i) has label **<=50K** vs **>50K**. We observe a feature vector $(\mathbf{x}_i)$ mixing quantitative and categorical inputs. We estimate the conditional probability $(\mathbb{P}(y=1 \mid \mathbf{x}))$.

3.2 Logistic regression

Logistic regression specifies $(\mathbb{P}(y=1 \mid \mathbf{x}) = \sigma(\eta(\mathbf{x})))$ where (σ) is the logistic function and (η) is linear in the transformed (encoded) features. In **sklearn**, **LogisticRegression** applies **L2 regularization** by default, stabilizing estimates when many dummy variables are created by one-hot encoding.

3.3 Random forest (*comparison model*)

A **random forest** averages predictions from many decision trees fit on bootstrap samples; it can capture nonlinearities and interactions. We include it as a simple, strong baseline to contrast with the linear logit.

3.4 Evaluation

On the fixed **test** partition we report **accuracy**, **ROC-AUC**, `classification_report`, the **confusion matrix**, and an **ROC curve**. We also compare against a **majority-class baseline** (always predict $\leq 50K$), since accuracy is misleading without it when classes are imbalanced.

3.5 Modeling choices and caveats

We keep both **education** and **education-num**. They encode similar information; including both lets the regularized logit down-weight redundancy, but it would also be defensible to drop one for a simpler story. **fnlwgt** is a Census sampling weight in the documentation; we keep it as an input because many Adult benchmarks do, but it is not a causal attribute of the person in the way age or occupation are.

4. Data

4.1 Source

The **Adult** data were donated to the [UCI Machine Learning Repository](#) (Becker & Kohavi). Files in `database/` mirror that distribution: `adult.data` (training), `adult.test` (test), and `adult.names` (variable descriptions).

4.2 Working directory

Open this notebook from the `final project` folder (same level as `database/`) so paths like `database/adult.data` resolve, matching how labs load `data/student_performance.csv` relative to the notebook location.

4.3 Packages

The next chunk imports libraries used across **Lab 4**, **Lab 5**, and **Assignment 1** (`numpy` , `pandas` , `matplotlib` , `seaborn` , `missingno`) plus `sklearn` for modeling.

```
In [12]: # Packages
import numpy as np
import pandas as pd
import missingno as msno
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from IPython.display import display

from sklearn.compose import ColumnTransformer
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    RocCurveDisplay,
    accuracy_score,
    classification_report,
    roc_auc_score,
)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

# Seaborn style (same as Lab 4 / Lab 5)
sns.set_style("whitegrid")
sns.set_palette("Set2")

rng = np.random.RandomState(42)
```

```
In [13]: from pathlib import Path

DATA_DIR = Path("database")
assert DATA_DIR.exists(), "Run from the `final project` folder so ./database

COLS = [
    "age",
    "workclass",
    "fnlwt",
    "education",
    "education-num",
    "marital-status",
    "occupation",
    "relationship",
    "race",
    "sex",
    "capital-gain",
    "capital-loss",
    "hours-per-week",
    "native-country",
    "income",
]

NUMERIC = [
    "age",
    "fnlwt",
    "education-num",
    "capital-gain",
```

```
    "capital-loss",
    "hours-per-week",
]
CATEGORICAL = [
    "workclass",
    "education",
    "marital-status",
    "occupation",
    "relationship",
    "race",
    "sex",
    "native-country",
]

def load_adult(data_dir: Path) -> tuple[pd.DataFrame, pd.DataFrame]:
    train = pd.read_csv(
        data_dir / "adult.data",
        names=COLS,
        skipinitialspace=True,
        na_values="?",
    )
    test = pd.read_csv(
        data_dir / "adult.test",
        names=COLS,
        skipinitialspace=True,
        na_values="?",
        skiprows=1,
    )
    test["income"] = test["income"].str.replace(".", "", regex=False).str.strip()
    return train, test

train_df, test_df = load_adult(DATA_DIR)
```

4.4 First look (shape , info , head)

As in the labs, we inspect dimensions, dtypes, and the first rows before modeling.

```
In [14]: print(train_df.shape, test_df.shape)
print(train_df.info())
train_df.head()
```

```

(32561, 15) (16281, 15)
<class 'pandas.DataFrame'>
RangeIndex: 32561 entries, 0 to 32560
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   age                   32561 non-null  int64
1   workclass             30725 non-null  str
2   fnlwgt               32561 non-null  int64
3   education             32561 non-null  str
4   education-num         32561 non-null  int64
5   marital-status        32561 non-null  str
6   occupation            30718 non-null  str
7   relationship          32561 non-null  str
8   race                 32561 non-null  str
9   sex                  32561 non-null  str
10  capital-gain          32561 non-null  int64
11  capital-loss          32561 non-null  int64
12  hours-per-week        32561 non-null  int64
13  native-country        31978 non-null  str
14  income                32561 non-null  str
dtypes: int64(6), str(9)
memory usage: 3.7 MB
None

```

Out[14]:

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife

4.5 Target encoding and feature matrices

We encode the label as 1 if `income == '>50K'` and 0 otherwise, then drop `income`

from predictors.

```
In [15]: LABEL_POS = ">50K"

def encode_target(s: pd.Series) -> pd.Series:
    return (s.str.strip() == LABEL_POS).astype(int)

y_train = encode_target(train_df["income"])
y_test = encode_target(test_df["income"])

X_train = train_df.drop(columns=["income"])
X_test = test_df.drop(columns=["income"])

print("Positive rate - train:", round(y_train.mean(), 4))
print("Positive rate - test:", round(y_test.mean(), 4))
```

Positive rate - train: 0.2408

Positive rate - test: 0.2362

4.6 Missing values

UCI codes unknown categories as `?`; `read_csv` maps those to `NaN`. Below we visualize **patterns of missingness** on the training predictors using `missingno`, which Lab 4 / Lab 5 import for this purpose.

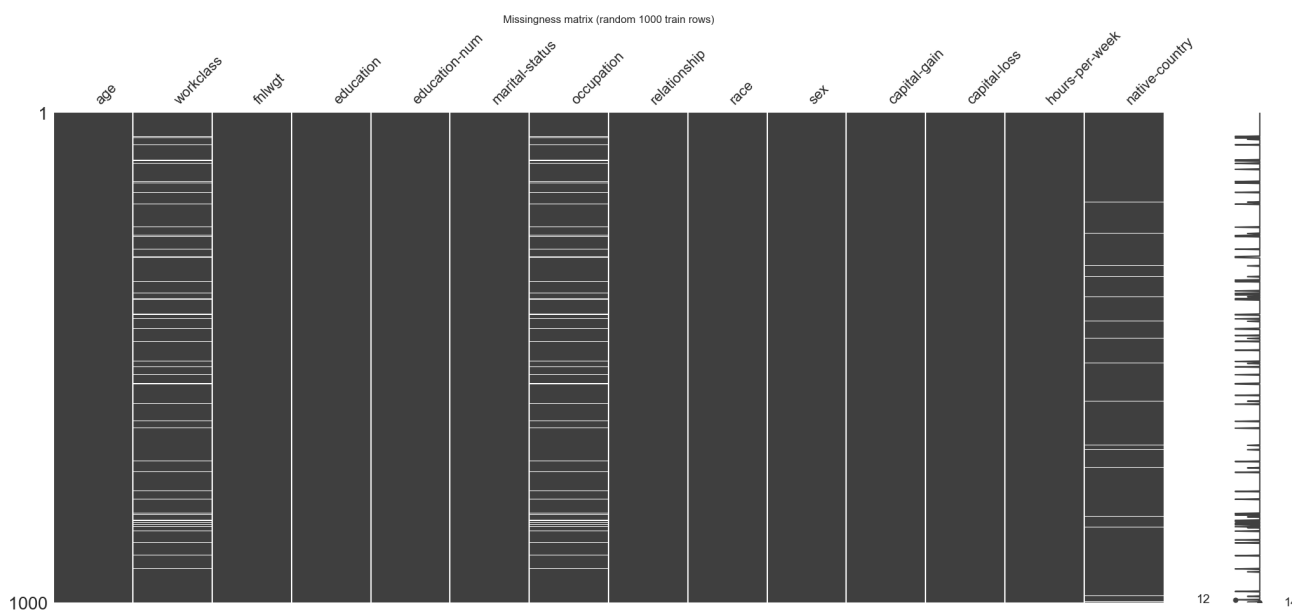
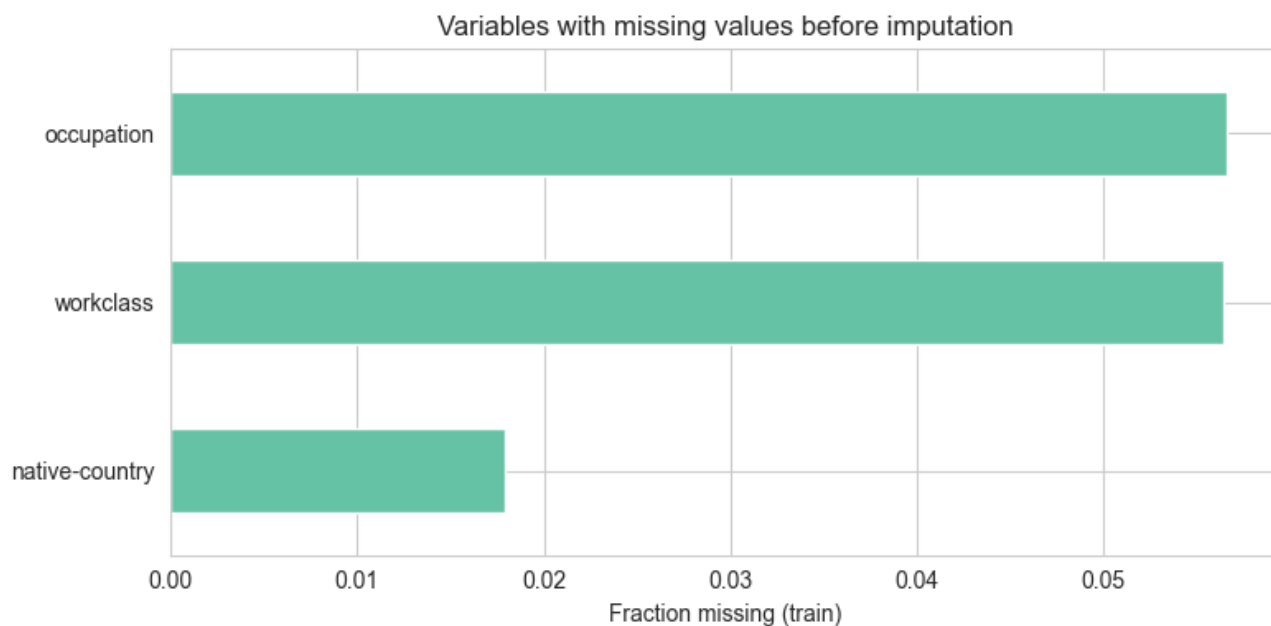
For modeling we **impute** inside `sklearn` pipelines (median for numeric features, most frequent for categorical) so test-set values are never used when fitting transforms on the training data.

```
In [16]: missing_frac = X_train.isna().mean().sort_values(ascending=False)
display(missing_frac[missing_frac > 0].to_frame("fraction_missing"))

plt.figure(figsize=(8, 4))
missing_frac[missing_frac > 0].sort_values().plot(kind="barh", color=sns.color_palette("magma"))
plt.xlabel("Fraction missing (train)")
plt.title("Variables with missing values before imputation")
plt.tight_layout()
plt.show()

# Missingness overview (same library stack as course labs)
msno.matrix(X_train.sample(min(1000, len(X_train)), random_state=42))
plt.title("Missingness matrix (random 1000 train rows)")
plt.show()
```

	fraction_missing
occupation	0.056601
workclass	0.056386
native-country	0.017905



4.7 Duplicates

Lab exercises emphasize checking duplicate rows after cleaning.

```
In [17]: print("Duplicate rows - train:", X_train.duplicated().sum())
```



```
print("Duplicate rows - test:", X_test.duplicated().sum())
```

Duplicate rows - train: 25

Duplicate rows - test: 6

4.8 Exploratory analysis

We mirror **Lab 5** themes: grouped comparisons and multivariate plots among quantitative variables. A full `pairplot` on tens of thousands of rows is slow; we plot a **random subsample** for visualization only.

```
In [18]: eda = train_df.copy()
eda["high_income"] = encode_target(eda["income"]).map({0: "<=50K", 1: ">50K"})

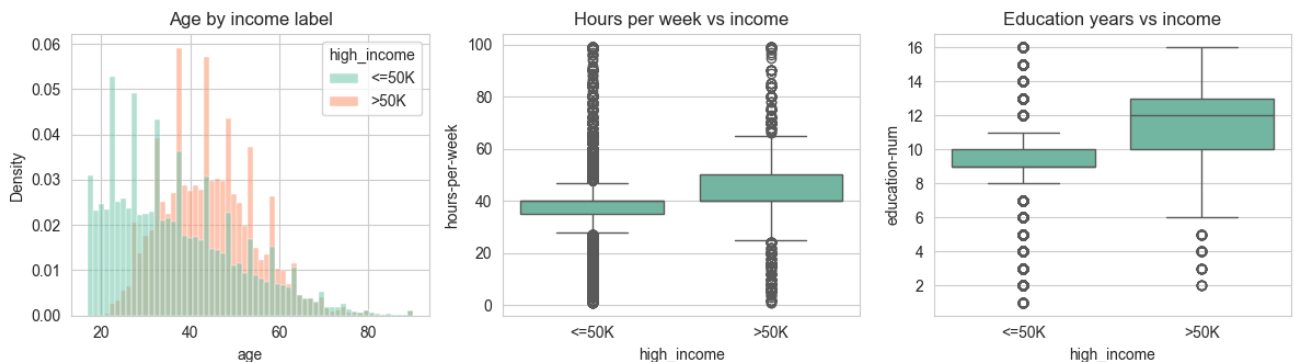
fig, axes = plt.subplots(1, 3, figsize=(12, 3.5))

sns.histplot(data=eda, x="age", hue="high_income", stat="density", common_norm=False)
axes[0].set_title("Age by income label")

sns.boxplot(data=eda, x="high_income", y="hours-per-week", ax=axes[1])
axes[1].set_title("Hours per week vs income")

sns.boxplot(data=eda, x="high_income", y="education-num", ax=axes[2])
axes[2].set_title("Education years vs income")

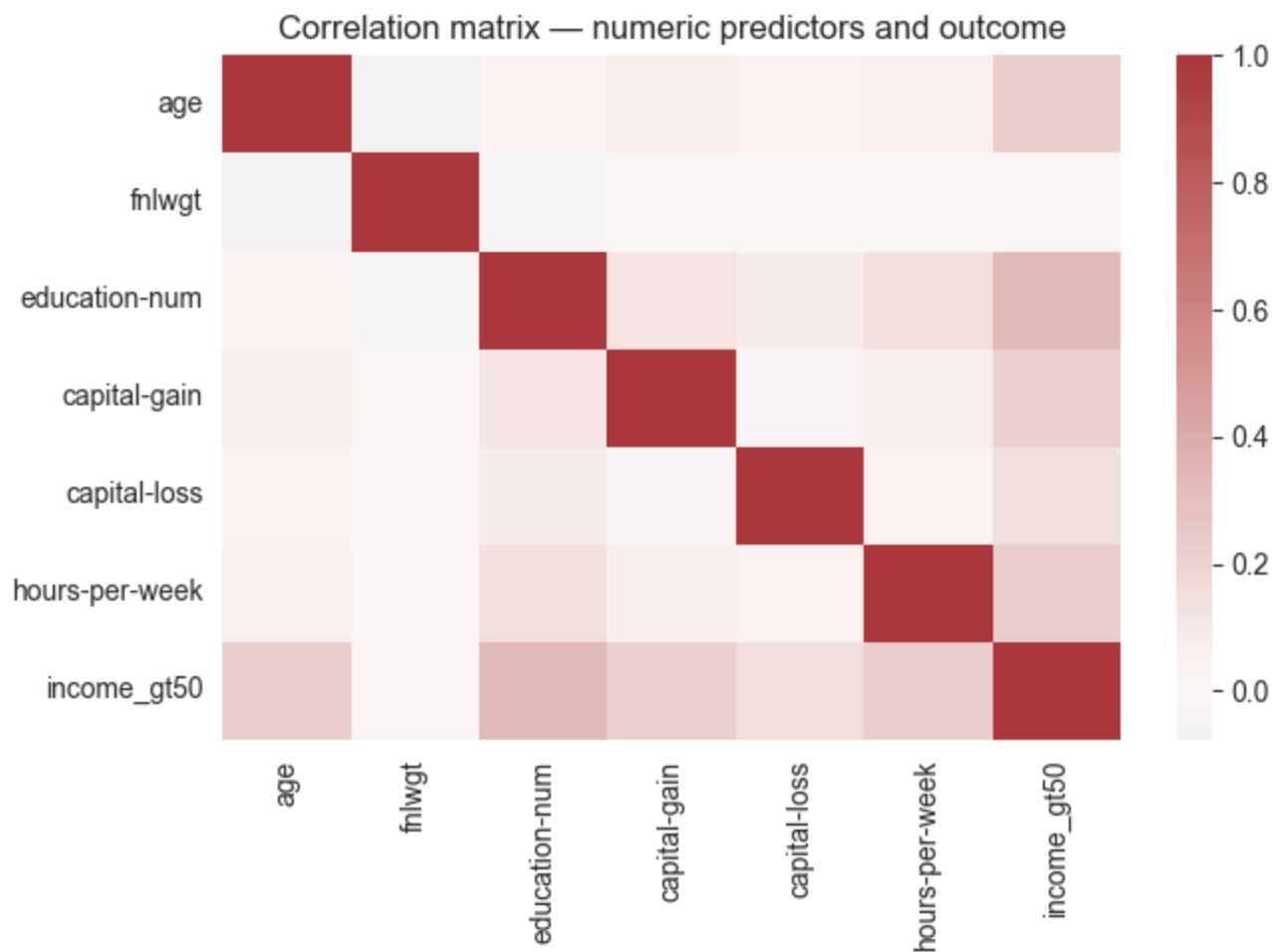
plt.tight_layout()
plt.show()
```



```
In [19]: num_eda = train_df[NUMERIC].copy()
num_eda["income_gt50"] = y_train.values
corr_matrix = num_eda.corr(numeric_only=True)

plt.figure(figsize=(7, 5))
sns.heatmap(corr_matrix, cmap="vlag", center=0)
plt.title("Correlation matrix - numeric predictors and outcome")
plt.tight_layout()
plt.show()
```

```
sample = eda.sample(n=min(1500, len(eda)), random_state=42)
pair_vars = [c for c in NUMERIC if c != "fnlwgt"]
g = sns.pairplot(sample, vars=pair_vars, hue="high_income", corner=True, diag_kind="kde")
g.figure.suptitle("Pairplot (1500-row subsample; fnlwgt omitted for readability)")
plt.show()
```



Pairplot (1500-row subsample; fnlwgt omitted for readability)



In the exploratory plots, people labeled **>50K** tend to report **more hours per week** and **more years of education** on average than those labeled **<=50K**, while **age** overlaps heavily between groups but shifts somewhat toward older workers among higher earners. The correlation heatmap aligns with that story: **education-num** and **hours-per-week** have clearer positive association with the binary high-income indicator than variables like **capital-gain**, which are sparse but can spike for some rows. The pairplot on a subsample is noisy, yet it reinforces that separation between labels is gradual rather than a single clean cutoff in one variable alone.

5. Results

5.0 Majority-class baseline

Roughly **76%** of test rows are **<=50K**. A classifier that always predicts **<=50K** therefore gets about **76.4%** accuracy without learning anything. Any serious model should beat that by a clear margin.

5.1 Pipelines

We build two pipelines that share the same imputation and one-hot encoding for categoricals. Logistic regression **additionally scales** numeric inputs; the forest sees **imputed but unscaled** numeric columns, which is standard because tree splits are scale-invariant.

```
In [20]: numeric_pipe = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]
)

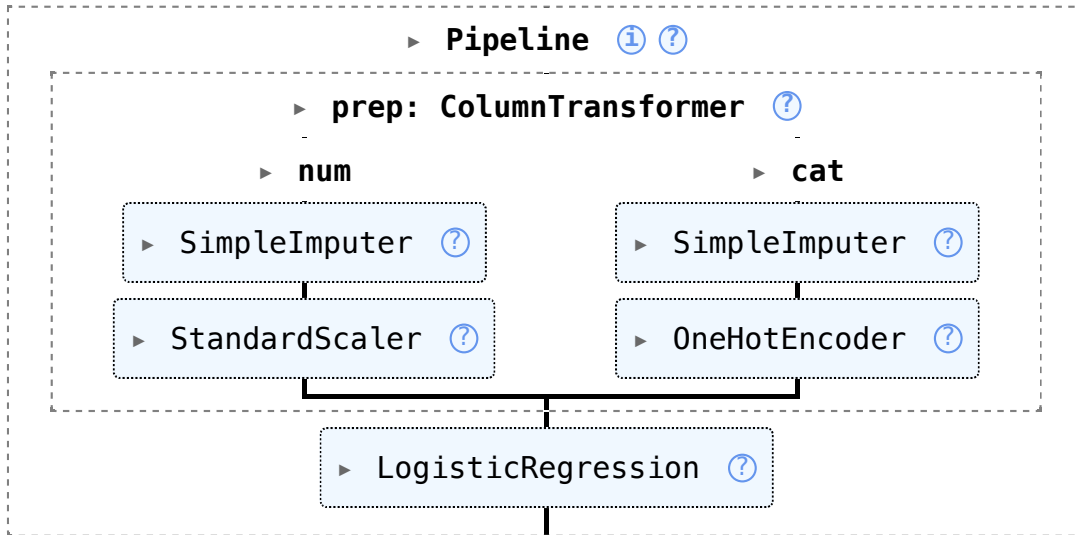
categorical_pipe = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
    ]
)

preprocess = ColumnTransformer(
    transformers=[
        ("num", numeric_pipe, NUMERIC),
        ("cat", categorical_pipe, CATEGORICAL),
    ]
)

log_reg = Pipeline(
    steps=[
        ("prep", preprocess),
        ("clf", LogisticRegression(max_iter=2000, random_state=42, solver="lbfgs")),
    ]
)

log_reg.fit(X_train, y_train)
```

Out[20]:



```

In [21]: def evaluate_model(name: str, model, X_te, y_te):
    proba = model.predict_proba(X_te)[:, 1]
    pred = (proba >= 0.5).astype(int)
    acc = accuracy_score(y_te, pred)
    auc = roc_auc_score(y_te, proba)
    maj = max(y_te.mean(), 1 - y_te.mean())
    print(f"=== {name} ===")
    print(f"Accuracy: {acc:.4f} (majority baseline {maj:.4f}) | ROC-AUC: {auc:.4f}")
    print(classification_report(y_te, pred, target_names=["<=50K", ">50K"]))

    fig, axes = plt.subplots(1, 2, figsize=(10, 4))
    ConfusionMatrixDisplay.from_predictions(y_te, pred, ax=axes[0], cmap="Blues")
    axes[0].set_title(f"{name} - confusion matrix")
    RocCurveDisplay.from_predictions(y_te, proba, ax=axes[1])
    axes[1].set_title(f"{name} - ROC curve")
    plt.tight_layout()
    plt.show()

    evaluate_model("Logistic regression", log_reg, X_test, y_test)
  
```

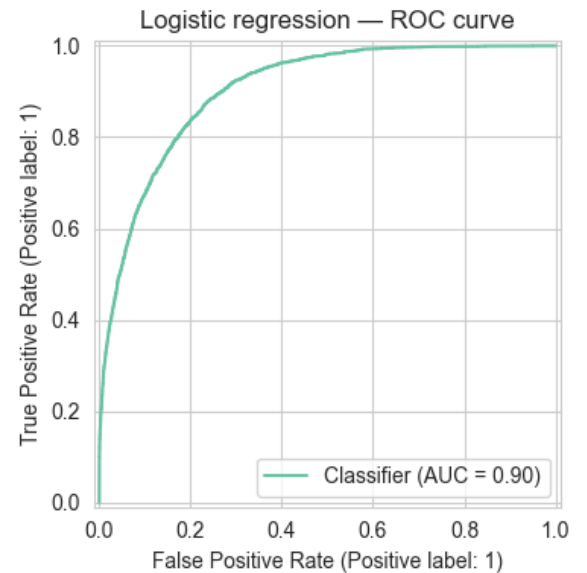
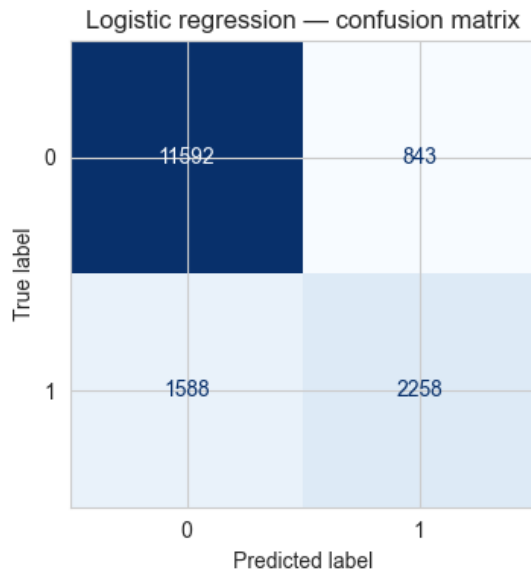
```

=== Logistic regression ===
  
```

```

Accuracy: 0.8507 (majority baseline 0.7638) | ROC-AUC: 0.9038
  
```

	precision	recall	f1-score	support
<=50K	0.88	0.93	0.91	12435
>50K	0.73	0.59	0.65	3846
accuracy			0.85	16281
macro avg	0.80	0.76	0.78	16281
weighted avg	0.84	0.85	0.84	16281



```
In [22]: rf_preprocess = ColumnTransformer(
    transformers=[
        ("num", SimpleImputer(strategy="median"), NUMERIC),
        (
            "cat",
            Pipeline(
                steps=[
                    ("imputer", SimpleImputer(strategy="most_frequent")),
                    ("onehot", OneHotEncoder(handle_unknown="ignore", sparse
                ]
            ),
            CATEGORICAL,
        ),
    ]
)

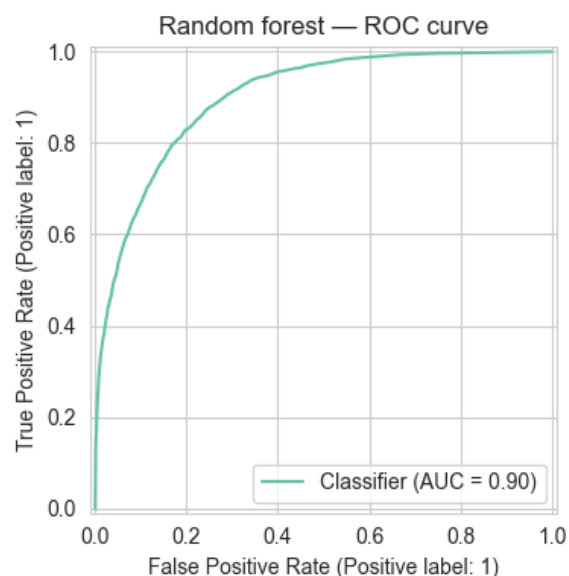
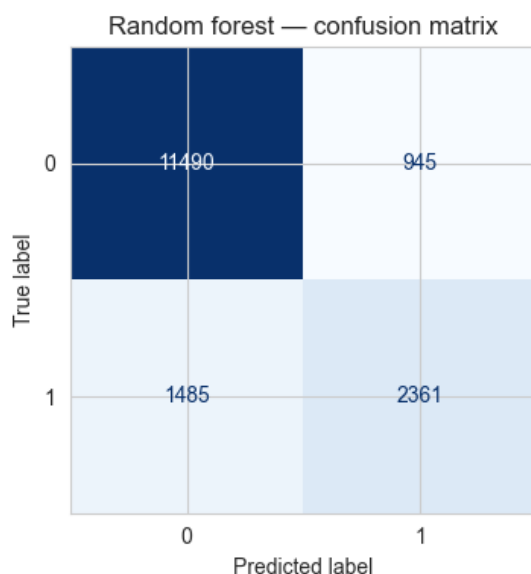
rf_clf = Pipeline(
    steps=[
        ("prep", rf_preprocess),
        (
            "clf",
            RandomForestClassifier(
                n_estimators=200,
                random_state=42,
                n_jobs=-1,
            ),
        ),
    ]
)

rf_clf.fit(X_train, y_train)
evaluate_model("Random forest", rf_clf, X_test, y_test)
```

=== Random forest ===

Accuracy: 0.8507 (majority baseline 0.7638) | ROC-AUC: 0.9008

	precision	recall	f1-score	support
<=50K	0.89	0.92	0.90	12435
>50K	0.71	0.61	0.66	3846
accuracy			0.85	16281
macro avg	0.80	0.77	0.78	16281
weighted avg	0.85	0.85	0.85	16281



5.2 Model comparison

On this fixed split, **logistic regression** and the **random forest** achieve **similar test accuracy** near **85%**, both well above the **76.4%** majority baseline. **ROC-AUC** is high for both; in our run the logit is slightly ahead on AUC while the forest is not automatically better despite extra flexibility. For the minority **>50K** class, check

`classification_report` and the confusion matrices: errors are mostly false negatives (predict $\leq 50K$ when truth is $> 50K$), which is expected when positives are rare.

The forest adds hyperparameters and interpretation cost. Here it does not clearly dominate the logit, so a course-appropriate takeaway is that **a simple linear probability model already captures most of the predictable signal** on this benchmark, and fancier methods need to earn their keep on validation metrics you care about.

6. Conclusion

This notebook reproduced a standard Adult workflow: clean missing tokens, explore associations, then fit and compare classifiers on the official test fold. **Education and hours worked** show consistent separation in exploratory views; formal models lift accuracy by roughly **9 percentage points** over always guessing the majority class. Results remain **associational**: they describe labels in a 1994-era extract, not causal effects of changing education or occupation today.

Limitations include **historical data**, **duplicate education encodings**, using `fnlwgt` as **a feature** even though it is really a survey weight, and **class imbalance** that makes naive accuracy easy to inflate. A natural extension would be **probability calibration** or **cost-sensitive thresholds** if false negatives for >50K are more expensive than false positives.

7. References

1. Becker, B., & Kohavi, R. (1996). Adult [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5XW20>
 2. Kohavi, R. (1996). Scaling Up the Accuracy of Naive-Bayes Classifiers: a Decision-Tree Hybrid. In *Proceedings of KDD-96* (secondary citation from UCI `adult.names` notes).
 3. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
-